

Informe de Laboratorio 04

Tema: JavaScript - Lógica y Manipulación del DOM

Nota

Estudiante	Escuela	Asignatura
Pumacayo Pinto Santos Nicolás spumacayop@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Introducción al Desarrollo Web Semestre: II Código: 2501210

Laboratorio	Tema	Duración
04	JavaScript - Lógica y Manipulación del DOM	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	29/09/2026	30/09/2026

1. Objetivos

- Aplicar sintaxis moderna de JavaScript (ES6+): variables let/const, funciones flecha, destructuring y métodos de arrays.
- Seleccionar, crear, modificar y eliminar nodos dinámicamente en el Document Object Model (DOM).
- Capturar y procesar eventos de usuario mediante Event Listeners y validar datos de entrada.
- Implementar almacenamiento persistente en el navegador cliente utilizando la API localStorage y la serialización JSON.

2. Equipos, materiales y temas utilizados

- Sistema Operativo Microsoft Windows 11 / GNU Linux.
- Visual Studio Code (VS Code).
- Navegador Web Google Chrome / Firefox con Herramientas de Desarrollo (DevTools Console).
- Git y GitHub.
- Bootstrap 5 (CDN).
- Estándares ECMAScript 6+ y API de DOM/LocalStorage.

3. URL de Repositorio Github

- URL del Repositorio GitHub principal:
- <https://github.com/NicolasUnsa/IDWebLab>
- URL para el laboratorio 04 en el Repositorio GitHub:
- <https://github.com/NicolasUnsa/IDWebLab/tree/main/Lab04>

4. Desarrollo de la Práctica

4.1. Marco Conceptual

JavaScript es el lenguaje de programación estándar interpretado en el cliente web. El **DOM (Document Object Model)** es una interfaz de programación que representa la estructura del documento como un árbol de nodos (objetos). Mediante métodos como `document.querySelector()`, `createElement()` y `appendChild()`, JavaScript puede alterar dinámicamente el contenido HTML y las clases CSS.

La interacción del usuario dispara **eventos** (p. ej. click, submit, input). La invocación de métodos `addEventListener()` asigna funciones escuchadoras (listeners). Adicionalmente, el almacenamiento persistente en el navegador cliente se logra mediante la API `localStorage`, guardando cadenas formateadas mediante `JSON.stringify()` y consumiéndolas con `JSON.parse()`.

4.2. Evolución y Transformación Paso a Paso

Para cumplir con las especificaciones de la aplicación "Gestor de Tareas Universitarias", el código base fue modificado gradualmente:

1. **Estructura de Datos:** Se transformó el arreglo plano de strings en un arreglo de objetos complejos con la estructura `{ id, titulo, curso, fechaEntrega, completada }`. Se empleó `Date.now()` para generar identificadores únicos.
2. **Captura y Validación de Datos:** Se añadieron inputs para `Curso` y "Fecha de Entrega" en el formulario. Al presionar el botón de envío, se intercepta el evento con `e.preventDefault()`, validando que no existan campos vacíos y que la fecha de entrega no sea menor a la fecha actual. En caso de error, se renderiza una alerta Bootstrap dinámica.
3. **Manipulación Dinámica y Delegación de Eventos:** Se eliminaron los atributos inline (`onclick`) y se asignó un único evento `click` en el contenedor principal (`#todo-list`). Con ello se detecta si se presionó "Eliminar" o "Completar", cambiando dinámicamente las clases del elemento (efecto de tachado).
4. **Filtros Visuales:** Se agregaron botones para filtrar tareas ("Todas", "Pendientes", "Completadas") procesando el arreglo con el método iterativo ES6+ `.filter()`.
5. **Persistencia en LocalStorage:** Toda mutación (agregar, cambiar estado, eliminar) invoca a una función de sincronización con `localStorage`, asegurando que al reabrir el navegador mediante el evento `DOMContentLoaded` las tareas se reconstruyan.

4.3. Código Fuente Completo

Listing 1: index.html - Estructura del Gestor de Tareas Universitarias

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gestor de Tareas Universitarias</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
    rel="stylesheet">
</head>
<body class="bg-light">

<div class="container mt-5">
  <div class="row justify-content-center">
    <div class="col-md-9">

      <div class="card shadow mb-4">
        <div class="card-header bg-primary text-white">
          <h1 class="h3 mb-0">Gestor de Tareas Universitarias</h1>
        </div>

        <div class="card-body">
          <div id="alert-container"></div>

          <form id="todo-form" class="mb-4">
            <div class="row g-3">
              <div class="col-md-5">
                <label for="todo-input" class="form-label
                  font-weight-bold">Titulo de la Tarea</label>
                <input type="text" id="todo-input" class="form-control"
                  placeholder="Ej. Informe de Laboratorio" autocomplete="off">
              </div>
              <div class="col-md-4">
                <label for="todo-course" class="form-label">Curso</label>
                <input type="text" id="todo-course" class="form-control"
                  placeholder="Ej. Desarrollo Web" autocomplete="off">
              </div>
              <div class="col-md-3">
                <label for="todo-date" class="form-label">Fecha de
                  Entrega</label>
                <input type="date" id="todo-date" class="form-control">
              </div>
            </div>
            <div class="mt-3 text-end">
              <button type="submit" class="btn btn-primary px-4">Agregar
                Tarea</button>
            </div>
          </form>

          <hr>

          <div class="d-flex justify-content-between align-items-center mb-3">
            <h2 class="h5 mb-0">Lista de Tareas Academicas</h2>
            <div class="btn-group" role="group" id="filter-buttons">
              <button type="button" class="btn btn-outline-primary active"
                data-filter="all">Todas</button>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>
```

Listing 2: app.js - Lógica JS y Manipulación del DOM

Página 4

```

        if (currentFilter === 'completed') return task.completada;
        return true;
    });

    if (filteredTasks.length === 0) {
        list.innerHTML = '<li class="list-group-item text-center text-muted">No hay tareas
            para mostrar.</li>';
        return;
    }

    filteredTasks.forEach(task => {
        const li = document.createElement('li');
        li.className = 'list-group-item d-flex justify-content-between align-items-center
            ${task.completada ? 'bg-light' : ''}';

        const textStyle = task.completada ? 'text-decoration-line-through text-muted' : '';
        const badgeStyle = task.completada ? 'bg-secondary' : 'bg-info text-dark';

        li.innerHTML = `
            <div class="me-auto">
                <h6 class="mb-1 ${textStyle}">${task.titulo}</h6>
                <small class="text-muted">Curso: <strong>${task.curso}</strong> | Vence:
                    ${task.fechaEntrega}</small>
            </div>
            <div class="d-flex align-items-center gap-2">
                <span class="badge ${badgeStyle}">${task.completada ? 'Completada' :
                    'Pendiente'}</span>
                <button class="btn btn-sm ${task.completada ? 'btn-warning' : 'btn-success'}
                    toggle-btn" data-id="${task.id}">
                    ${task.completada ? 'Desmarcar' : 'Completar'}
                </button>
                <button class="btn btn-danger btn-sm delete-btn"
                    data-id="${task.id}">Eliminar</button>
            </div>
        `;
        list.appendChild(li);
    });
}

form.addEventListener('submit', (e) => {
    e.preventDefault();
    clearAlert();

    const titulo = inputTitulo.value.trim();
    const curso = inputCurso.value.trim();
    const fechaEntrega = inputFecha.value;

    if (!titulo || !curso || !fechaEntrega) {
        showAlert('Todos los campos son obligatorios.');
```

```
}

const newTask = {
  id: Date.now(),
  titulo,
  curso,
  fechaEntrega,
  completada: false
};

tasks.push(newTask);
saveTasks();
renderTasks();

form.reset();
});

list.addEventListener('click', (e) => {
  const target = e.target;
  const id = Number(target.getAttribute('data-id'));

  if (!id) return;

  if (target.classList.contains('delete-btn')) {
    tasks = tasks.filter(task => task.id !== id);
    saveTasks();
    renderTasks();
  }

  if (target.classList.contains('toggle-btn')) {
    const task = tasks.find(t => t.id === id);
    if (task) {
      task.completada = !task.completada;
      saveTasks();
      renderTasks();
    }
  }
});

filterButtons.addEventListener('click', (e) => {
  if (e.target.tagName === 'BUTTON') {
    document.querySelectorAll('#filter-buttons button').forEach(btn =>
      btn.classList.remove('active'));
    e.target.classList.add('active');

    currentFilter = e.target.getAttribute('data-filter');
    renderTasks();
  }
});

document.addEventListener('DOMContentLoaded', renderTasks);
```

4.4. Estructura del directorio del proyecto

```
lab04/
|--- index.html
|--- app.js
|--- InformeLab04.pdf
```

4.5. Evidencia de Ejecución

A continuación se adjuntan las capturas del funcionamiento del sistema web interactivo desarrollado:



Gestor de Tareas Universitarias

Título de la Tarea: Ej. Informe de Laboratorio | Curso: Ej. Desarrollo Web | Fecha de Entrega: dd/mm/aaaa

Agregar Tarea

Lista de Tareas Académicas | **Todas** | **Pendientes** | **Completadas**

Título de la Tarea	Curso	Vence	Estado	Acciones
Laboratorio04	IDWebLab	2026-10-01	Pendiente	Completar, Eliminar
RCU	Química	2026-10-08	Pendiente	Completar, Eliminar
TIF	Desarrollo Emocional	2026-10-17	Pendiente	Completar, Eliminar
Trabajo Grupal	Cálculo I	2026-10-05	Pendiente	Completar, Eliminar

Figura 1: Interfaz Principal del Gestor de Tareas Universitarias con tareas agregadas y filtros activos.



Gestor de Tareas Universitarias

¡Atención! Todos los campos son obligatorios.

Título de la Tarea: 123 | Curso: Ej. Desarrollo Web | Fecha de Entrega: 15/10/2026

Agregar Tarea

Lista de Tareas Académicas | **Todas** | **Pendientes** | **Completadas**

Título de la Tarea	Curso	Vence	Estado	Acciones
Laboratorio04	IDWebLab	2026-10-01	Completada	Desmarcar, Eliminar
TIF	Desarrollo Emocional	2026-10-17	Completada	Desmarcar, Eliminar

Figura 2: Validación dinámica del formulario con alerta de error ante datos inválidos o vacíos.

5. Cuestionario Teórico

1. ¿Cuál es la diferencia de comportamiento entre las palabras clave `var`, `let` y `const` respecto al alcance (scope) y al hoisting?

Respuesta:

- **Alcance (Scope):** `var` tiene un alcance de función o global. Si se declara dentro de un bloque (por ejemplo, dentro de un `if` o `for`), la variable escapa de dicho bloque. Por el contrario, `let` y `const` poseen alcance de **bloque** (`{}`), limitando su existencia y visibilidad al bloque exacto en el que fueron declaradas.
- **Hoisting:** Las tres declaraciones son elevadas (hoisted) al inicio de su contexto durante la fase de compilación. No obstante, `var` se inicializa automáticamente con `undefined`, permitiendo acceder a ella antes de su línea de código (retornando `undefined`). Por su parte, `let` y `const` se ubican en la llamada **Zona Muerta Temporal (TDZ)**, provocando un error de ejecución (`ReferenceError`) si se intentan usar antes de su declaración. Adicionalmente, `const` exige asignación obligatoria en su declaración y prohíbe reasignaciones.

2. Explique la técnica de Delegación de Eventos (Event Delegation) en el DOM y por qué es más eficiente que asignar listeners individuales a múltiples elementos.

Respuesta: La delegación de eventos aprovecha la propagación de eventos en fase de burbujeo (*Event Bubbling*). En lugar de asociar un listener a cada botón o elemento dinámico (por ejemplo, a cada botón de eliminar en un listado largo), se asigna **un único listener a un elemento ancestro o contenedor común** (como el `<ul>`). Cuando se activa un evento dentro de cualquier hijo, este sube en la jerarquía del DOM hasta el ancestro, donde se analiza mediante `event.target` qué elemento específico desencadenó la acción.

Es más eficiente por dos razones principales:

- a) **Ahorro de Memoria:** Se reduce la cantidad de listeners activos registrados en memoria.
 - b) **Soporte para Elementos Dinámicos:** Los nuevos elementos creados e insertados en el DOM dinámicamente en el futuro responderán automáticamente al evento sin necesidad de volver a vincular listeners.
3. ¿Por qué es necesario utilizar `JSON.stringify()` al almacenar objetos complejos en `localStorage` y qué limitaciones tiene este almacenamiento en términos de seguridad?

Respuesta:

- **Necesidad de `JSON.stringify()`:** La API `localStorage` solo puede almacenar pares clave-valor en formato de texto (*string*). Si intentamos almacenar directamente un objeto o arreglo, JavaScript ejecuta su método implícito `.toString()`, guardando únicamente la cadena `"[object Object]"`. Usar `JSON.stringify()` convierte el objeto en una cadena codificada en JSON que preserva su estructura para ser reconstruida con `JSON.parse()`.
- **Limitaciones de Seguridad:**
 - a) **Ataques XSS (Cross-Site Scripting):** Cualquier script inyectado maliciosamente en la aplicación puede acceder libremente al contenido de `localStorage` mediante `window.localStorage`.
 - b) **Texto Plano sin Encriptación:** Los datos se almacenan sin cifrar directamente en el navegador del usuario, por lo que nunca deben guardarse credenciales, tokens confidenciales o información privada sensible.

6. Rúbricas

6.1. Entregable Informe

Tabla 1: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y fácil de leer.

6.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la siguiente tabla:

Tabla 2: Niveles de desempeño

Puntos	Nivel			
	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 3: Rúbrica para contenido del Informe y demostración (Guía 04)

Criterio (según rúbrica de la guía)		Puntos	Checklist	Estudiante	Profesor
1. Manipulación del DOM y Eventos (35 %)	Crea, modifica y elimina nodos del DOM. Maneja eventos adecuadamente usando <code>e.preventDefault()</code> y delegación de eventos.	7	X	6	
2. Lógica JS y Métodos ES6+ (30 %)	Estructura el código utilizando <code>let/const</code> , arrow functions, destructuring y métodos de arreglos (<code>map</code> , <code>filter</code> , <code>find</code>).	6	X	5	
3. Persistencia LocalStorage (20 %)	Persiste y recupera el estado completo correctamente en <code>localStorage</code> y serialización JSON sin errores de inicialización.	4	X	3	
4. Cuestionario Teórico (15 %)	Responde con precisión y rigurosidad técnica sobre scope, hoisting, delegación de eventos y seguridad JSON.	3	X	2	
Total		20		16	

7. Referencias

- Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.
- MDN Web Docs. (2025). *Document Object Model (DOM) & Events*. Developer Mozilla.
- Simpson, K. (2015). *You Don't Know JS: ES6 & Beyond*. O'Reilly Media.